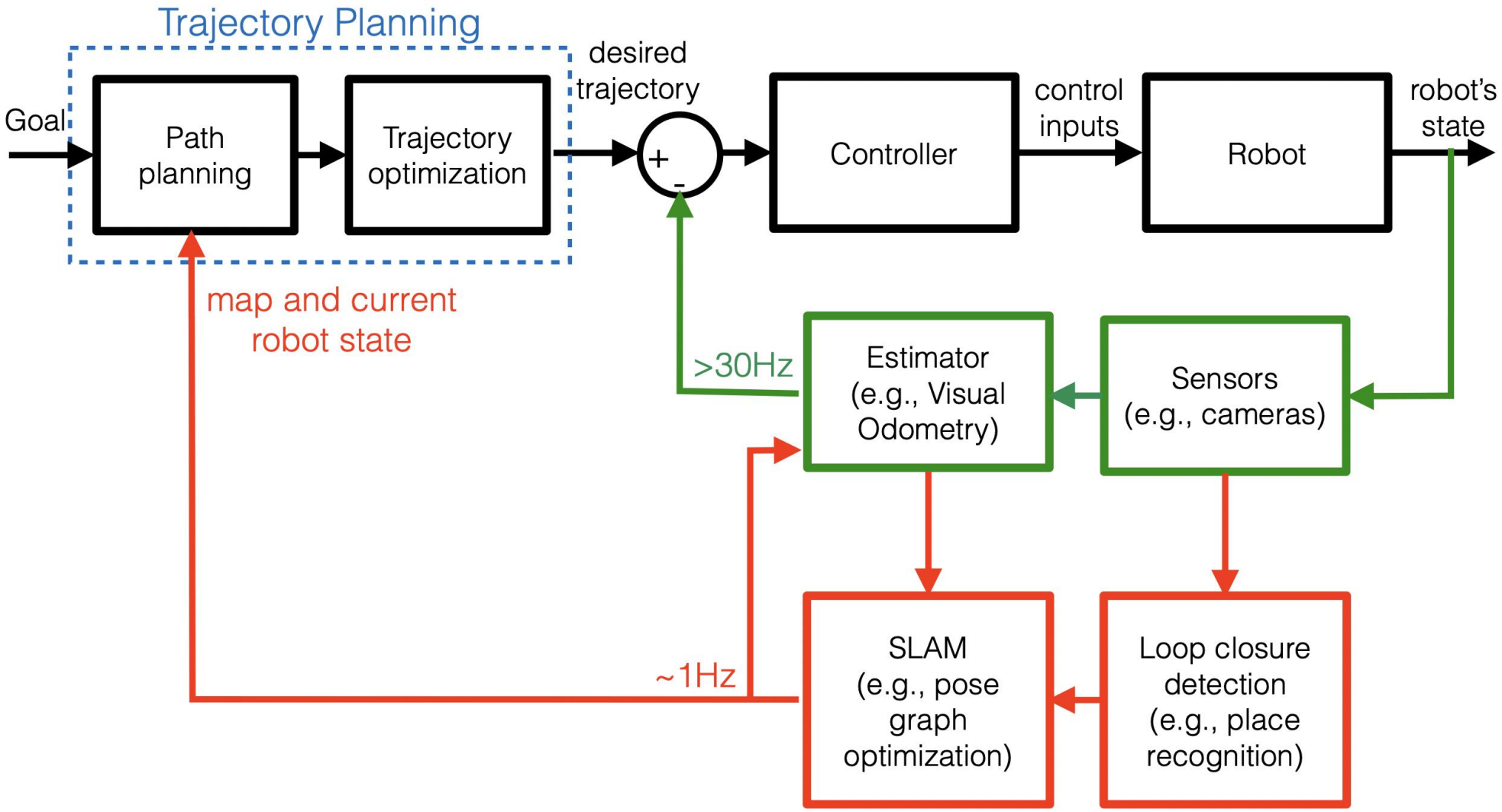
Visual Navigation for Autonomous Vehicles

Last Lecture – Overview

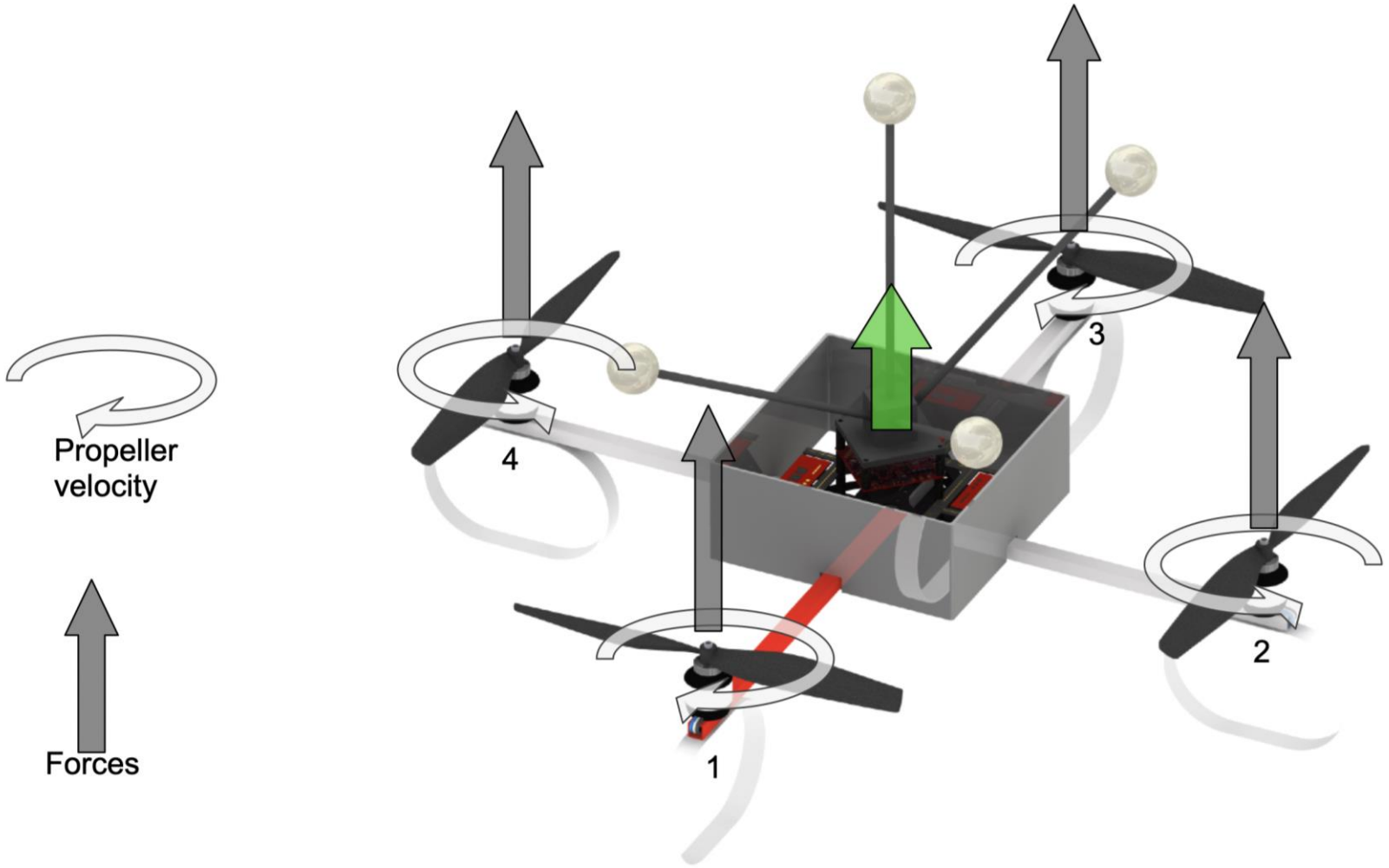
Self-driving Cars

Self-flying Drones

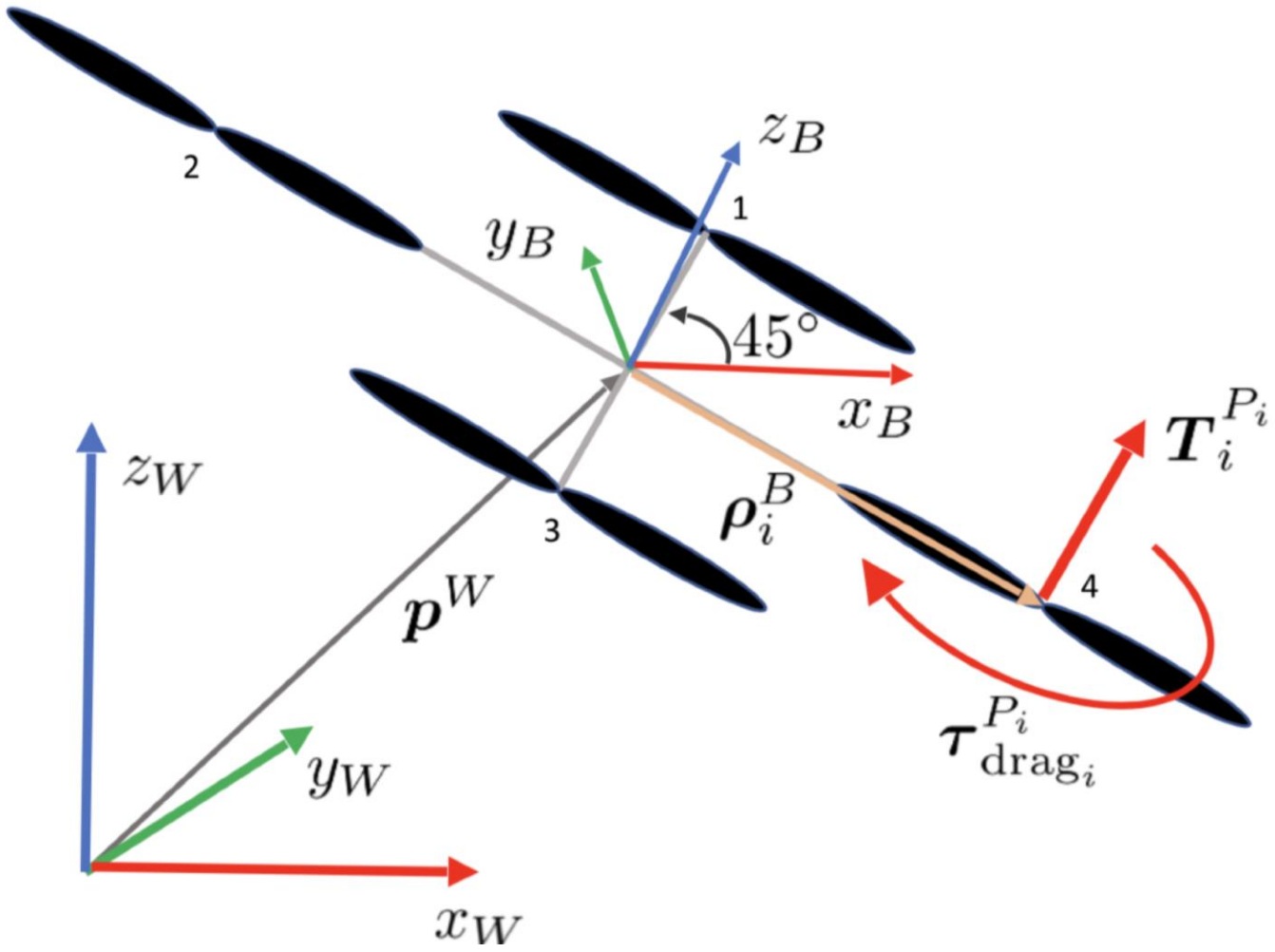
What we have covered in this course…



Quadrotor Dynamics



Quadrotor Dynamics



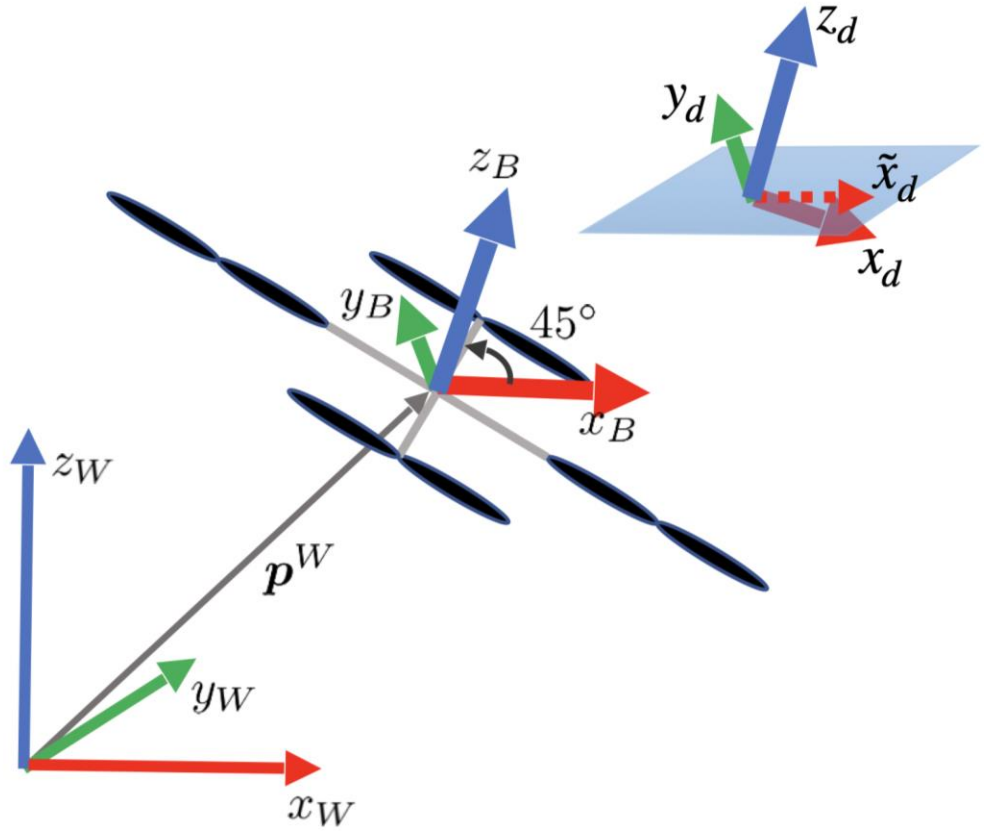
$$\begin{bmatrix} m\boldsymbol{a}^{\mathrm{W}} \\ \boldsymbol{\mathcal{J}}\boldsymbol{\alpha}^{\mathrm{B}} \end{bmatrix} = \begin{bmatrix} m\boldsymbol{g}^{\mathrm{W}} \\ -\boldsymbol{\omega}^{\mathrm{B}} \times \boldsymbol{\mathcal{J}}\boldsymbol{\omega}^{\mathrm{B}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{R}_{\mathrm{B}}^{\mathrm{W}} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I}_3 \end{bmatrix} \begin{bmatrix} \boldsymbol{f}_{thrust}^{\mathrm{B}} \\ \boldsymbol{\tau}_{drag}^{\mathrm{B}} + \boldsymbol{\tau}_{thrust}^{\mathrm{B}} \end{bmatrix} = \begin{bmatrix} -mg\boldsymbol{e}_3 \\ -\boldsymbol{\omega}^{\mathrm{B}} \times \boldsymbol{\mathcal{J}}\boldsymbol{\omega}^{\mathrm{B}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{R}_{\mathrm{B}}^{\mathrm{W}} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I}_3 \end{bmatrix} \boldsymbol{F}\boldsymbol{w}$$

$$\begin{bmatrix} f_x^{\mathrm{B}} \\ f_y^{\mathrm{B}} \\ f_z^{\mathrm{B}} \\ \tau_x^{\mathrm{B}} \\ \tau_y^{\mathrm{B}} \\ \tau_z^{\mathrm{B}} \end{bmatrix} = \boldsymbol{F}\boldsymbol{w} = \begin{bmatrix} c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 \\ c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_1^{\mathrm{B}} \times \boldsymbol{e}_3 & -c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_2^{\mathrm{B}} \times \boldsymbol{e}_3 & c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_3^{\mathrm{B}} \times \boldsymbol{e}_3 & -c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_4^{\mathrm{B}} \times \boldsymbol{e}_3 \end{bmatrix} \boldsymbol{w}$$

Geometric Controller

$$R_B^W = [x_B^W \; y_B^W \; z_B^W]$$

$$R_d^W = [x_d^W \; y_d^W \; z_d^W]$$



Geometric Controller

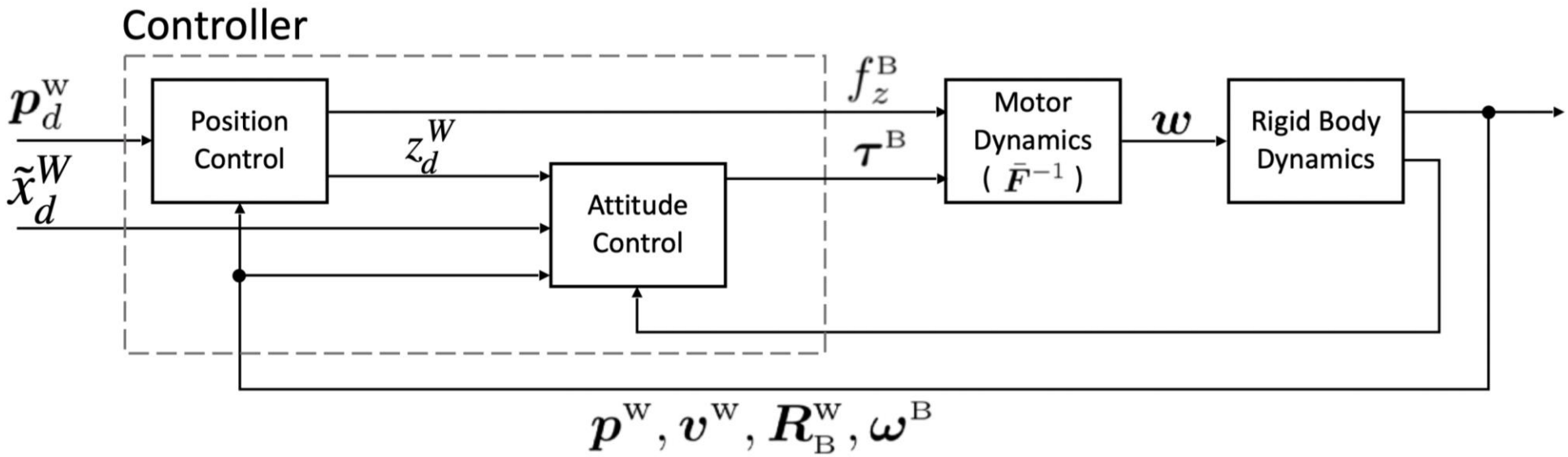
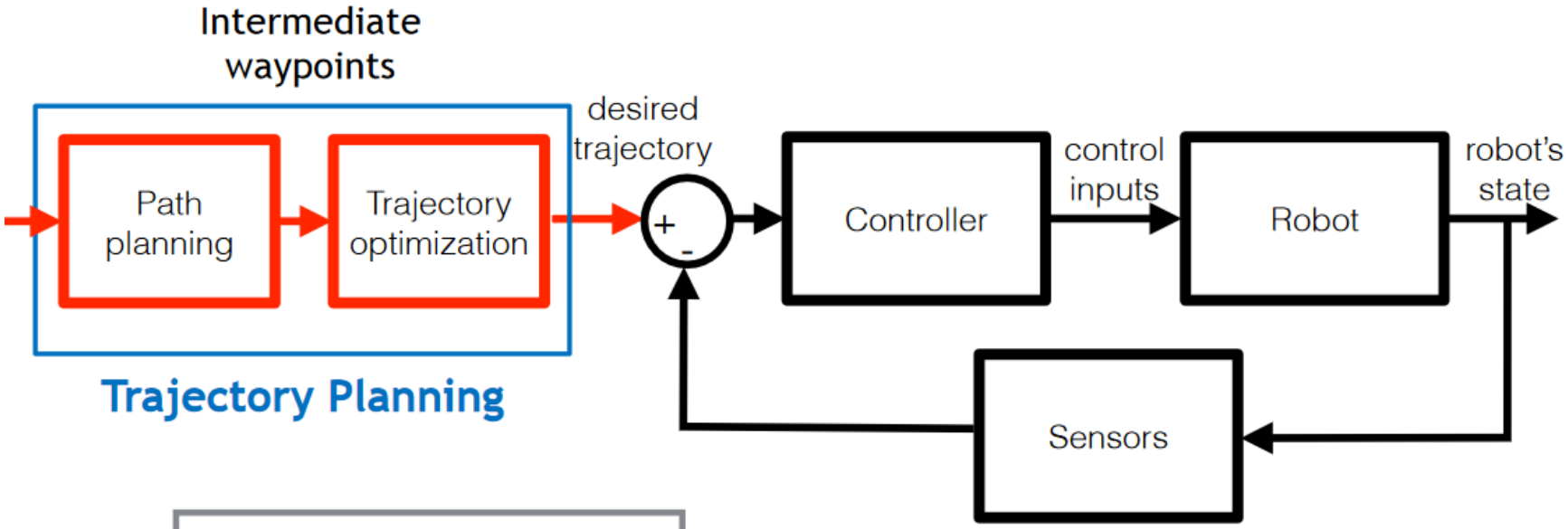
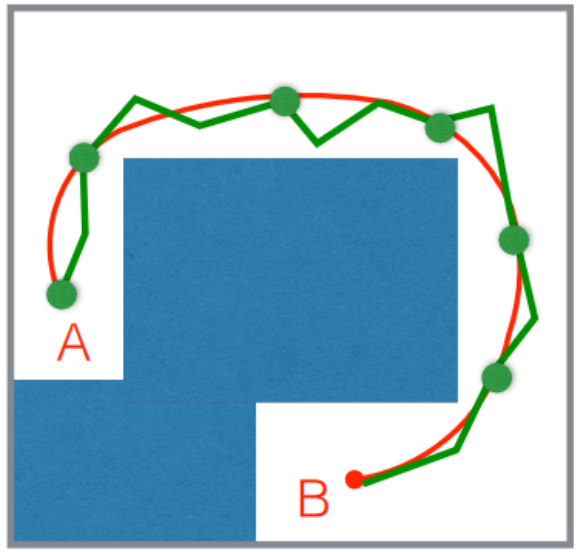


Figure 7.2: The geometric control loops for position and attitude control.

$$\begin{bmatrix} f_x^{\mathrm{B}} \\ f_y^{\mathrm{B}} \\ f_z^{\mathrm{B}} \\ \tau_x^{\mathrm{B}} \\ \tau_y^{\mathrm{B}} \\ \tau_z^{\mathrm{B}} \end{bmatrix} = \boldsymbol{F}\boldsymbol{w} = \begin{bmatrix} c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 & c_f\boldsymbol{e}_3 \\ c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_1^{\mathrm{B}} \times \boldsymbol{e}_3 & -c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_2^{\mathrm{B}} \times \boldsymbol{e}_3 & c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_3^{\mathrm{B}} \times \boldsymbol{e}_3 & -c_d\boldsymbol{e}_3 + c_f\boldsymbol{\rho}_4^{\mathrm{B}} \times \boldsymbol{e}_3 \end{bmatrix} \boldsymbol{w} \quad , \omega = F^{-1}T$$

Trajectory Optimization





Trajectory Optimization

$$
\begin{aligned}
\min_{\boldsymbol{x}(t),\boldsymbol{u}(t)} \quad & J(\boldsymbol{u}(t),\boldsymbol{x}(t)) \\
\text{subject to} \quad & \dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t),\boldsymbol{u}(t)) \\
& \boldsymbol{x}(0) = \bar{\boldsymbol{x}}_0 \\
& \boldsymbol{x}(T) = \bar{\boldsymbol{x}}_T \\
& \boldsymbol{u}_{min} \leq \boldsymbol{u}(t) \leq \boldsymbol{u}_{max} \\
& \boldsymbol{x}_{min} \leq \boldsymbol{x}(t) \leq \boldsymbol{x}_{max}
\end{aligned}
$$

Trajectory optimization – flat output

$$J(\boldsymbol{\sigma}, \dot{\boldsymbol{\sigma}}, \ddot{\boldsymbol{\sigma}}, \ldots) = \int_0^T (x^{(4)}(t))^2 dt + \int_0^T y^{(4)}(t)^2 dt + \int_0^T z^{(4)}(t)^2 dt + \int_0^T \psi^{(2)}(t)^2 dt$$

$$\boldsymbol{\sigma}(0) = \boldsymbol{\sigma}_0 \qquad \text{(initial state)}$$

$$\boldsymbol{\sigma}^{(1)}(0) = \boldsymbol{\sigma}_0^{(1)} \qquad \text{(first derivative at starting point)}$$

$$\vdots$$

$$\boldsymbol{\sigma}^{(q)}(0) = \boldsymbol{\sigma}_0^{(q)} \qquad \text{(q-th derivative at starting point)}$$

$$\boldsymbol{\sigma}(t) = [x, y, z, \psi]^\mathsf{T}$$

$$\boldsymbol{\sigma}(T) = \boldsymbol{\sigma}_T \qquad \text{(final state)}$$

$$\boldsymbol{\sigma}^{(1)}(T) = \boldsymbol{\sigma}_T^{(1)} \qquad \text{(first derivative at end point)}$$

$$\vdots$$

$$\boldsymbol{\sigma}^{(q)}(T) = \boldsymbol{\sigma}_T^{(q)} \qquad \text{(q-th derivative at end point)}$$

Trajectory Optimization –Euler Lagrange method

- Assumption: polynomial with fixed degree

$$P(t) = p_N t^N + p_{N-1} t^{N-1} + \ldots + p_2 t^2 + p_1 t + p_0$$

$$\begin{aligned}
\underset{P(t)}{\text{minimize}} \quad & \int_0^T (P^{(r)}(t))^2 dt \\
\text{subject to} \quad & P(0) = x_0 \\
& \vdots \\
& P^{(q)}(0) = x_0^{(q)} \\
& P(T) = x_T \\
& \vdots \\
& P^{(q)}(T) = x_T^{(q)}
\end{aligned}$$

$$\frac{\partial \mathcal{L}}{\partial x} - \frac{d}{dt}\frac{\partial \mathcal{L}}{\partial \dot{x}} = 0$$

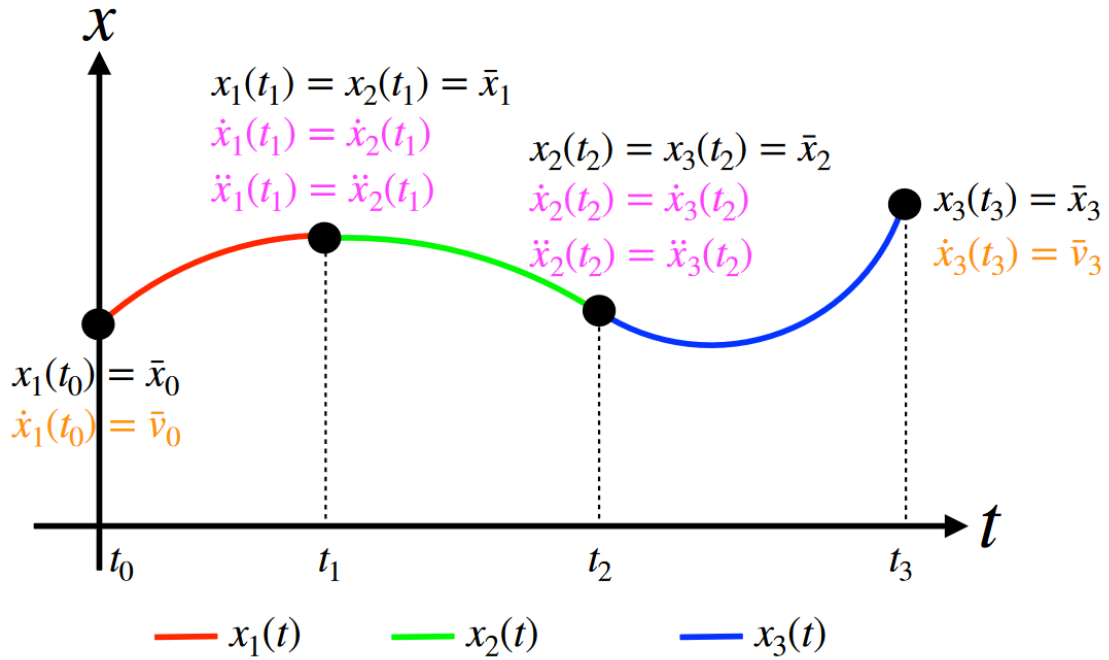
$$\sum_{k=0}^{r} (-1)^k \frac{d^k}{dt^k}\left(\frac{\partial \mathcal{L}}{\partial x^{(k)}}\right) = 0$$

Minimum Snap Trajectory Optimization

$$\frac{d^4}{dt^4}\left(\frac{\partial \mathcal{L}}{\partial P^{(4)}}\right) = 0 \Rightarrow \frac{d^4}{dt^4}\left(2P^{(4)}(t)\right) = 0 \Rightarrow \frac{d^8}{dt^8}P(t) = 0$$

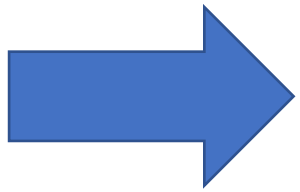
$$P(t) = p_7 t^7 + p_6 t^6 + p_5 t^5 + p_4 t^4 + \ldots + p_1 t^1 + p_0$$

$$\mathbf{A_{8\times 8}}\, \boldsymbol{p} = \mathbf{b_{8\times 1}}$$



QP-based Trajectory Optimization

$$\begin{aligned}
\underset{P(t)}{\text{minimize}} \quad & \int_0^T (P^{(r)}(t))^2 dt \\
\text{subject to} \quad & P(0) = \bar{x}_0 \\
& \vdots \\
& P^{(q)}(0) = x_0^{(q)} \\
& P(T) = \bar{x}_T \\
& \vdots \\
& P^{(q)}(T) = x_T^{(q)}
\end{aligned}$$



$$\begin{aligned}
\min \quad & \boldsymbol{p}^\top \boldsymbol{Q} \boldsymbol{p} \\
\text{subject to} \quad & \boldsymbol{A}\boldsymbol{p} = \boldsymbol{b}
\end{aligned}$$

$$P(t) = p_N t^N + p_{N-1} t^{N-1} + \ldots + p_2 t^2 + p_1 t + p_0$$

$$\begin{aligned}
\underset{P(t)}{\text{minimize}} \quad & \int_0^T (P^{(r)}(t))^2 dt \\
\text{subject to} \quad & P(0) = x_0 \\
& \vdots \\
& P^{(q)}(0) = x_0^{(q)} \\
& P(T) = x_T \\
& \vdots \\
& P^{(q)}(T) = x_T^{(q)}
\end{aligned}$$

$$J = \boldsymbol{p}^\mathsf{T} \boldsymbol{Q} \boldsymbol{p} = \sum_{l=0}^{N} \sum_{k=0}^{N} Q_{lk} p_l p_k$$

$$Q_{lk} = \begin{cases} \prod_{m=0}^{r-1} (l-m)(k-m) \frac{T^{l+k-2r+1}}{l+k-2r+1} & l \geq r \wedge k \geq r \\ 0 & l < r \vee k < r \end{cases}, \quad \text{and } Q_{kl} = Q_{lk}$$

Camera Pinhole Model

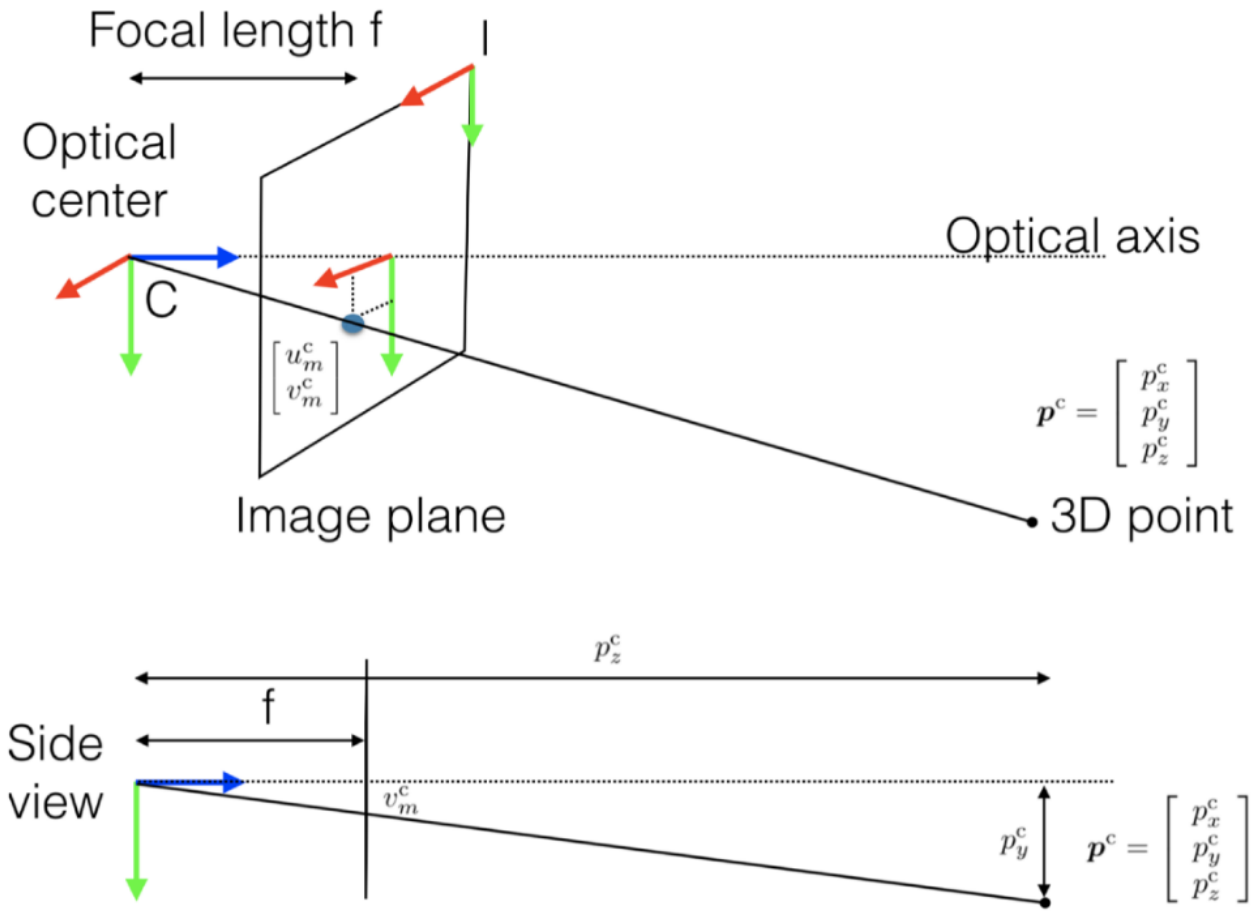
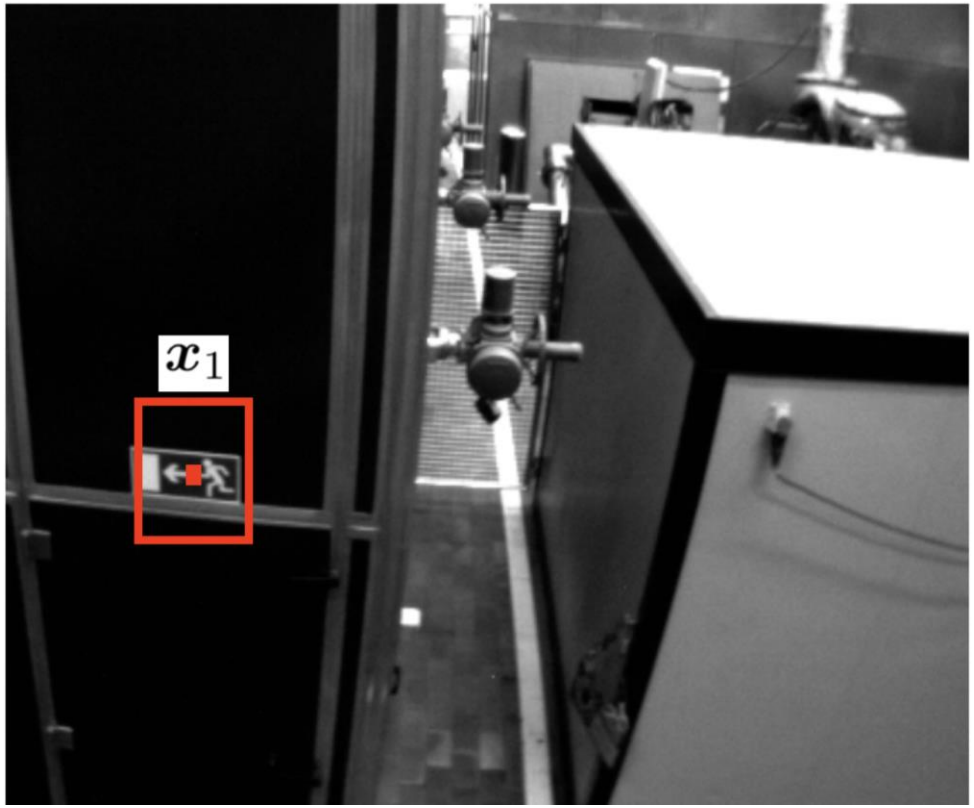


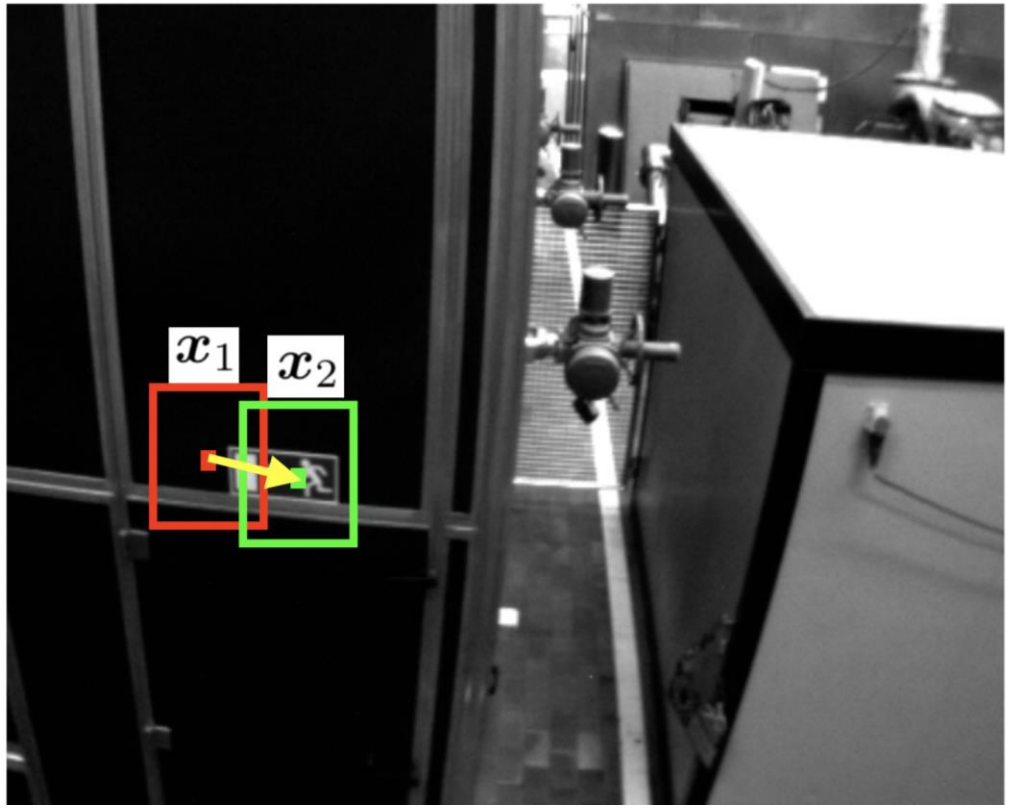
Figure 11.1: Pinhole Model.

$$p_z^{\mathrm{c}} \begin{bmatrix} u_m^{\mathrm{c}} \\ v_m^{\mathrm{c}} \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_x^{\mathrm{c}} \\ p_y^{\mathrm{c}} \\ p_z^{\mathrm{c}} \end{bmatrix}$$

$$p_z^{\mathrm{c}} \begin{bmatrix} u^{\mathrm{I}} \\ v^{\mathrm{I}} \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} s_x f & s_\theta f & o_x \\ 0 & s_y f & o_y \\ 0 & 0 & 1 \end{bmatrix}}^{\text{intrinsic calibration}} \overbrace{\begin{bmatrix} \boldsymbol{R}_{\mathrm{w}}^{\mathrm{c}} & \boldsymbol{t}_{\mathrm{w}}^{\mathrm{c}} \end{bmatrix}}^{\text{extrinsic calibration}} \quad \tilde{\boldsymbol{p}}^{\mathrm{w}} = \boldsymbol{\Pi} \tilde{\boldsymbol{p}}^{\mathrm{w}}$$

Feature Extraction and Tracking

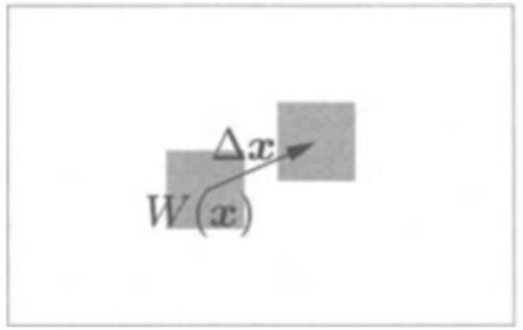


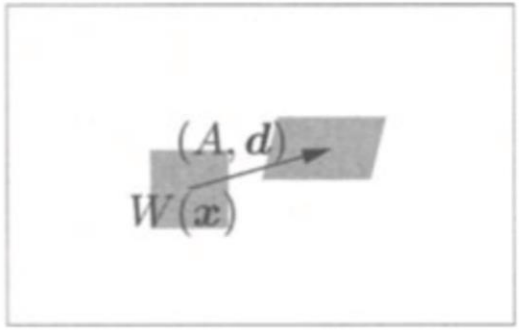


Feature Extraction and Tracking

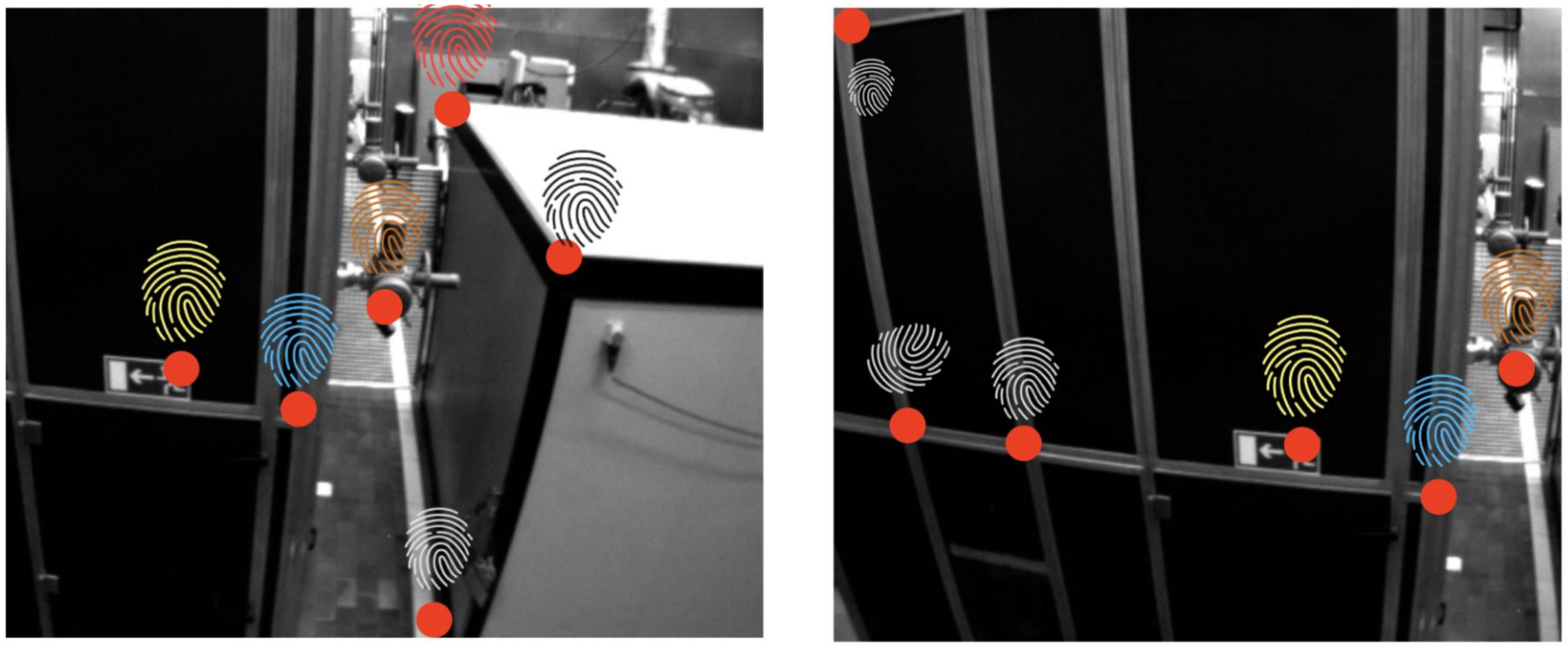
$$\min_{\boldsymbol{A},\boldsymbol{\delta}} \sum_{\boldsymbol{y}\in W(\boldsymbol{x}_1)} \|\mathcal{I}_1(\boldsymbol{y}) - \mathcal{I}_2(\boldsymbol{A}\boldsymbol{y}+\boldsymbol{\delta})\|^2$$

(affine motion model)



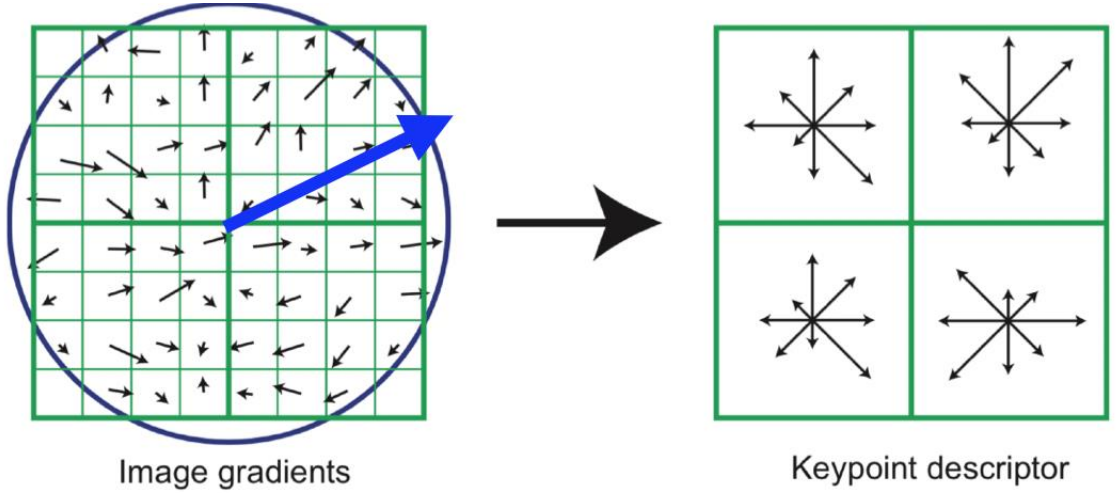


Descriptor-based Feature matching



Descriptor is a signature we attach to a (point) feature, that describes local appearance

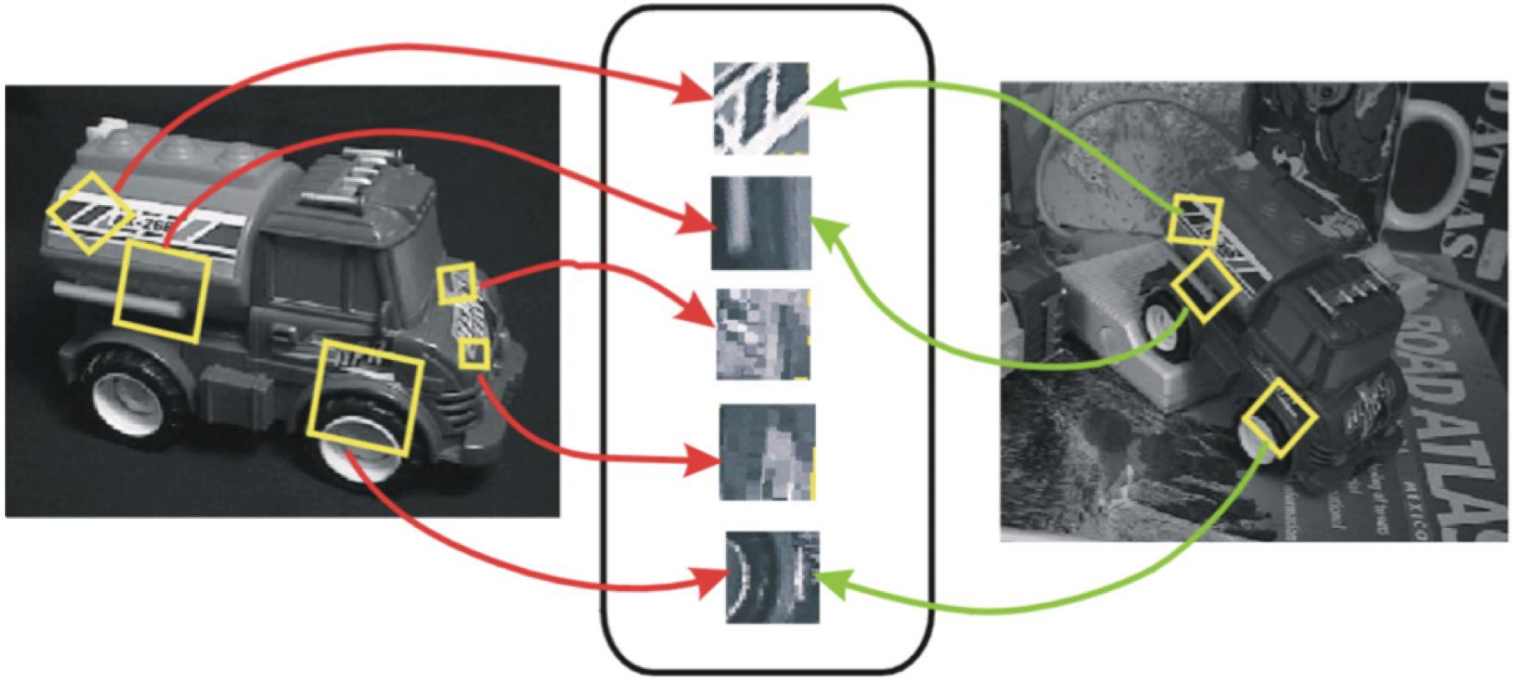
SIFT Descriptor



How to get SIFT descriptor?

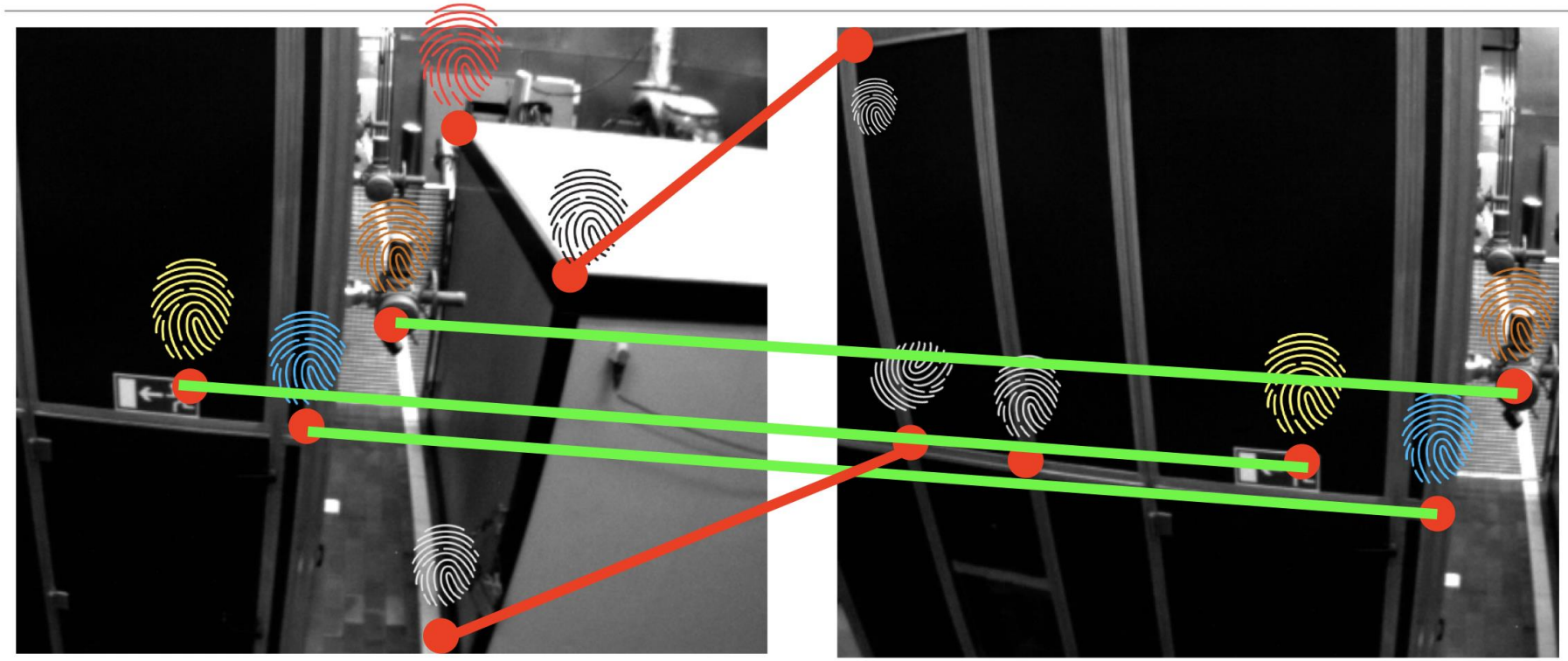
- Transform all gradients with respect to (main) orientation
- Split window in 16 squares and for each compute a histogram with 8 sectors
- Stack histogram into a **descriptor** vector of 16 x 8 = 128 scalars
- Normalize to have norm = 1

Feature Matching



- For each descriptor in I_1 find closest descriptor in I_2 (nearest neighbor)
- Speed up with **approximate** nearest neighbor algorithms (FLANN library)

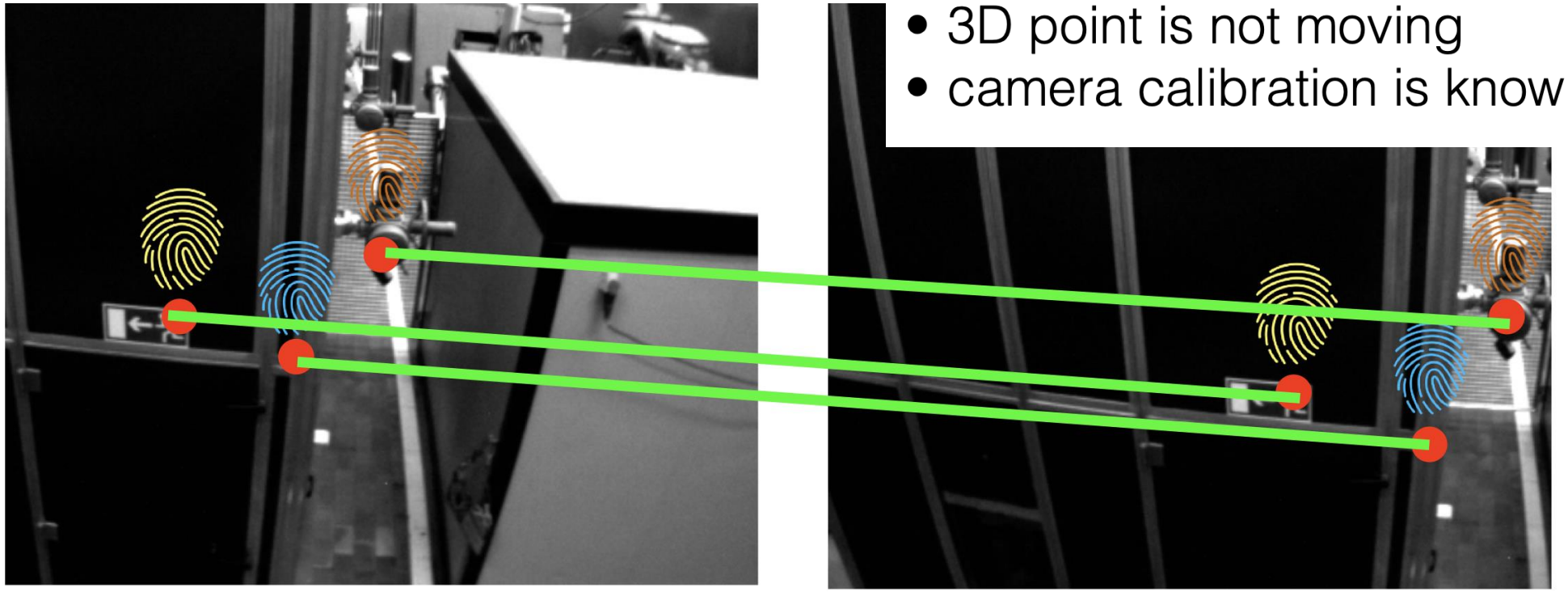
Pose Estimation: 2-view Geometry



Question: can we estimate the motion of the camera between I_1 and I_2 using pixel correspondences?

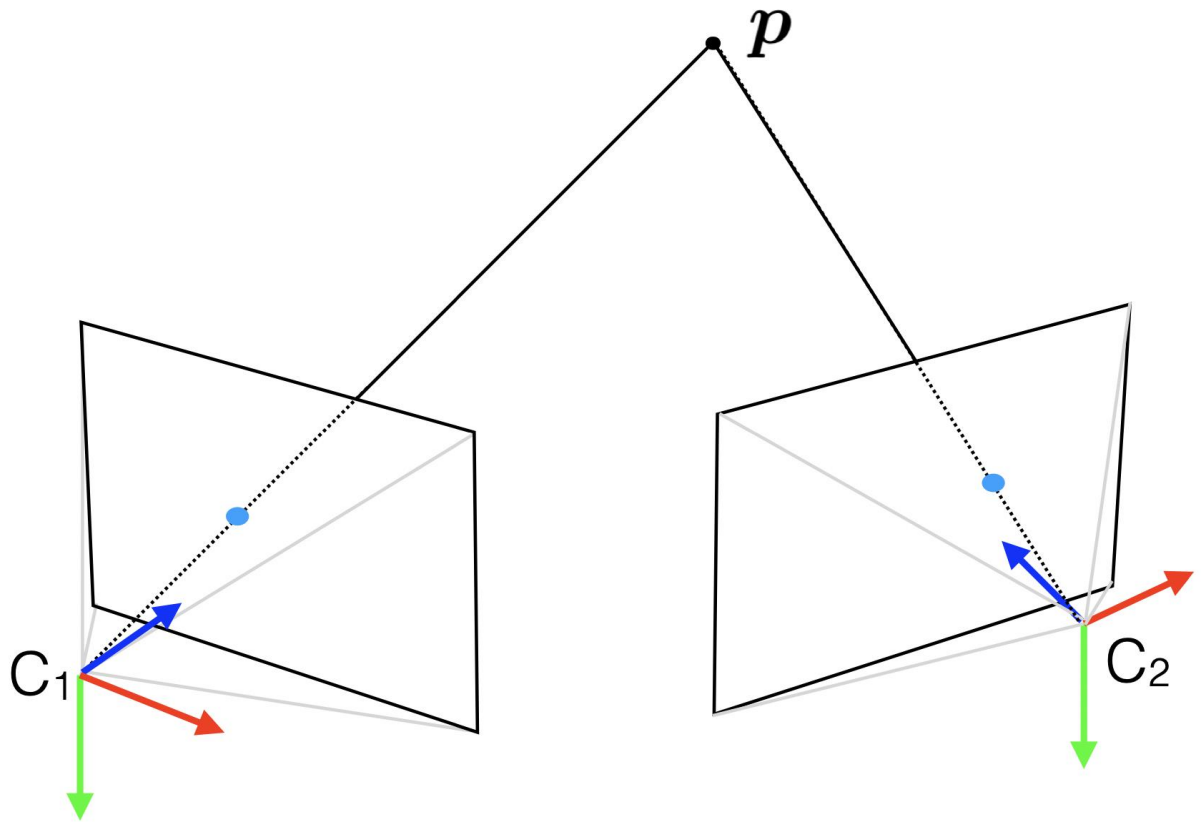
Pose Estimation: 2-view Geometry

- no wrong correspondences (outliers)
- 3D point is not moving
- camera calibration is known



Question: can we estimate the motion of the camera between I_1 and I_2 using pixel correspondences?

2-view Geometry



$$p_z^{\mathrm{C1}}\ \tilde{\boldsymbol{x}}_1 = \boldsymbol{K}_1\ [\boldsymbol{R}_{\mathrm{W}}^{\mathrm{C1}}\ \boldsymbol{t}_{\mathrm{W}}^{\mathrm{C1}}]\ \tilde{\boldsymbol{p}}^{\mathrm{W}}$$

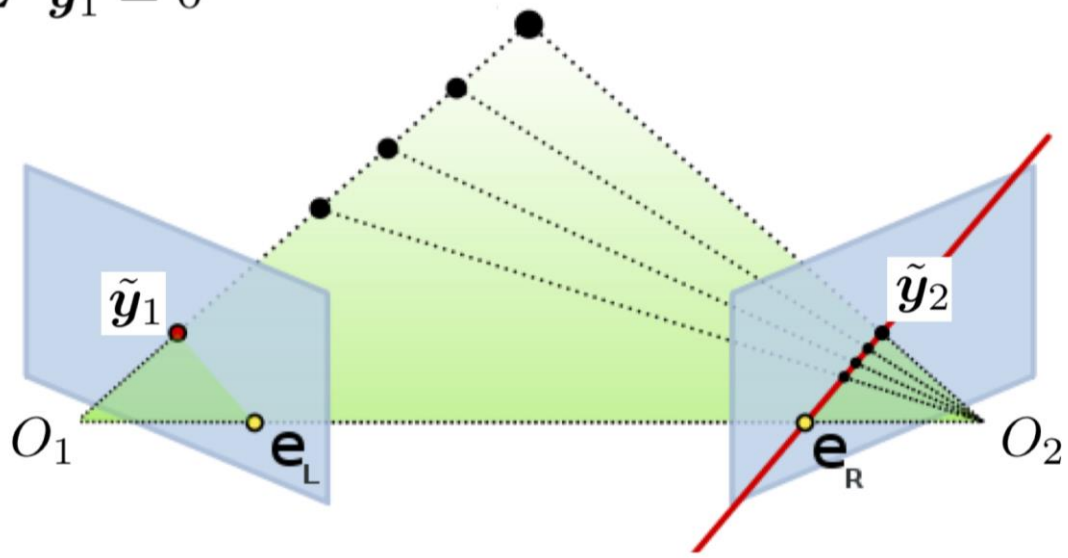
$$\boldsymbol{K}_1 = \begin{bmatrix} s_{x_1} f_1 & s_{\theta_1} f_1 & o_{x_1} \\ 0 & s_{y_1} f_1 & o_{y_1} \\ 0 & 0 & 1 \end{bmatrix}$$

$$p_z^{\mathrm{C2}}\ \tilde{\boldsymbol{x}}_2 = \boldsymbol{K}_2\ [\boldsymbol{R}_{\mathrm{W}}^{\mathrm{C2}}\ \boldsymbol{t}_{\mathrm{W}}^{\mathrm{C2}}]\ \tilde{\boldsymbol{p}}^{\mathrm{W}}$$

$$\boldsymbol{K}_2 = \begin{bmatrix} s_{x_2} f_2 & s_{\theta_2} f_2 & o_{x_2} \\ 0 & s_{y_2} f_2 & o_{y_2} \\ 0 & 0 & 1 \end{bmatrix}$$

Epipolar Geometry

$$\tilde{\boldsymbol{y}}_2^{\mathsf{T}} \; \boldsymbol{E} \; \tilde{\boldsymbol{y}}_1 = 0$$



2-view Geometry and Epipolar Equation

Given *N* calibrated pixel correspondences:

$$(\tilde{\boldsymbol{y}}_{1,k}, \tilde{\boldsymbol{y}}_{2,k}) \text{ for } k = 1, \ldots, N$$

1. leverage the epipolar constraints to estimate the essential matrix ***E***

$$\tilde{\boldsymbol{y}}_{2,k}^{\top} \boldsymbol{E}\, \tilde{\boldsymbol{y}}_{1,k} = 0$$

2. Retrieve the rotation and translation (up to scale) from the ***E***

$$\boldsymbol{E} = [\boldsymbol{t}]_{\times} \boldsymbol{R}$$

Estimation of poses from correspondence

Given *N* calibrated pixel correspondences:

$$(\tilde{\boldsymbol{y}}_{1,k}, \tilde{\boldsymbol{y}}_{2,k}) \text{ for } k = 1, \ldots, N$$

1. leverage the epipolar constraints to estimate the essential matrix ***E***

$$\tilde{\boldsymbol{y}}_{2,k}^{\mathsf{T}} \boldsymbol{E} \tilde{\boldsymbol{y}}_{1,k} = 0$$

For 8 points: $\boldsymbol{A}\boldsymbol{e} = 0$ N>8 points: $\arg\min_{\|\boldsymbol{e}\|=1} \|\boldsymbol{A}\boldsymbol{e}\|^2$

2. Retrieve the rotation and translation (up to scale) from the ***E***

$$\boldsymbol{E} = [\boldsymbol{t}]_{\times} \boldsymbol{R}$$

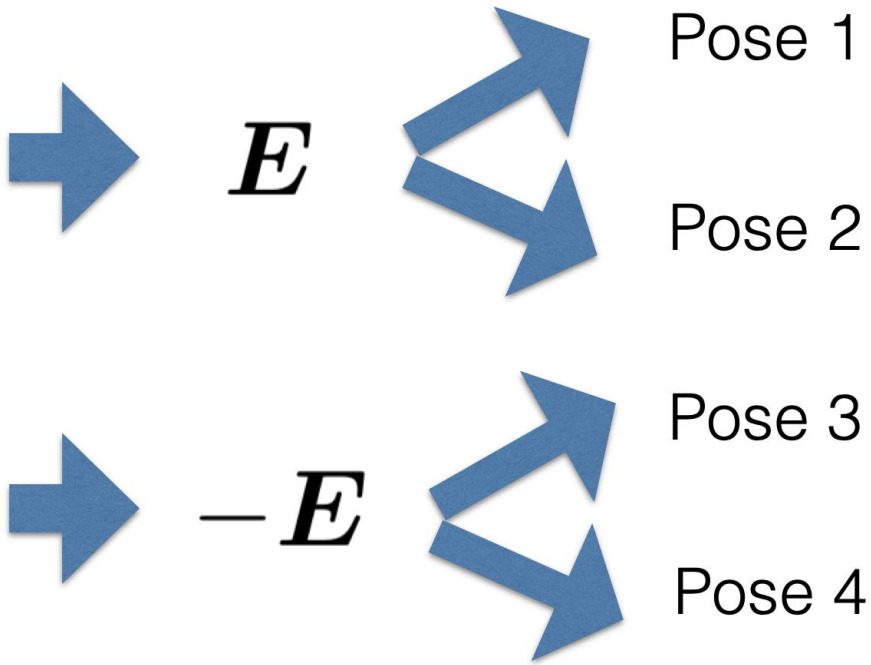
Retrieve Pose from Essential Matrix

Theorem 1 (Pose recovery from essential matrix, Thm 5.7 in [1]). *There exist exactly two relative poses* $(\boldsymbol{R}, \boldsymbol{t})$ *with* $\boldsymbol{R} \in \mathrm{SO}(3)$ *and* $\boldsymbol{t} \in \mathbb{R}^3$ *corresponding to a nonzero essential matrix* $\boldsymbol{E}$ *(i.e., such that* $\boldsymbol{E} = [\boldsymbol{t}]_\times \boldsymbol{R}$*):*

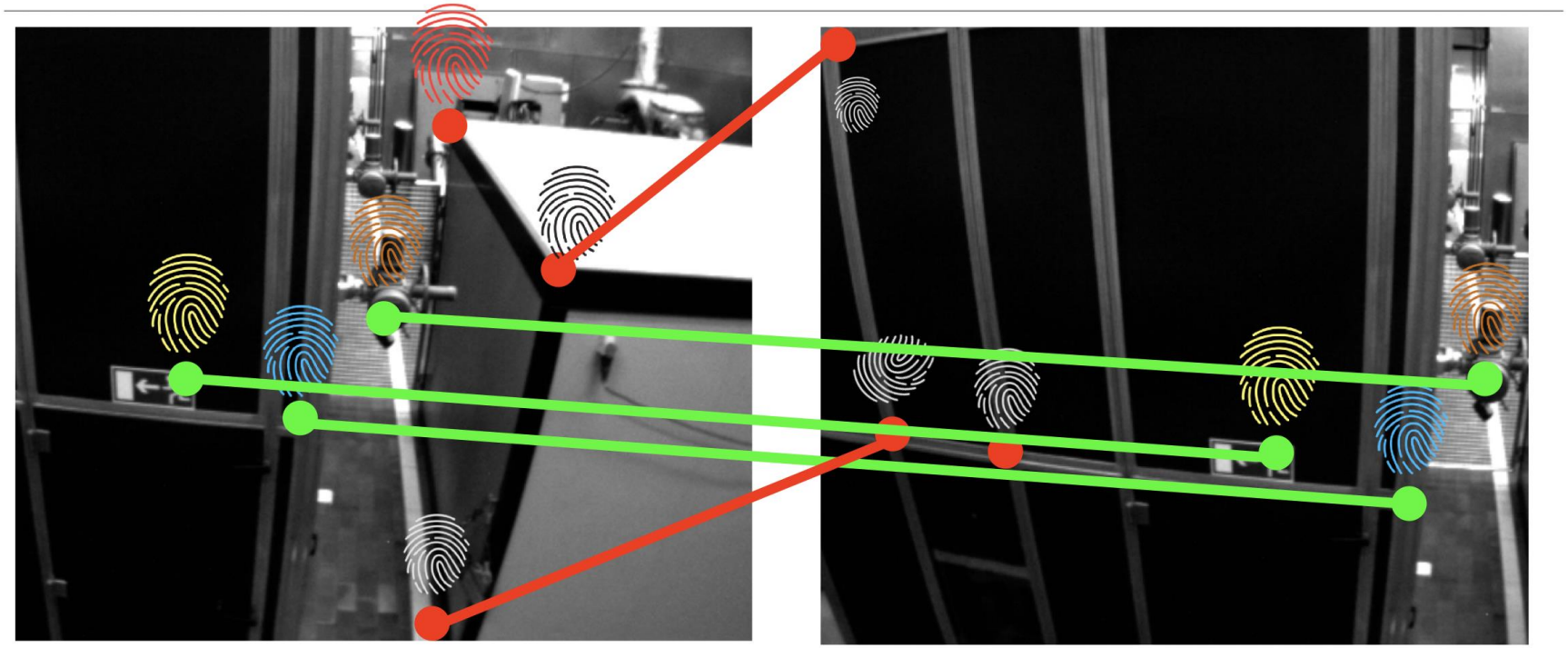
$$\boldsymbol{t}_1 = \boldsymbol{U}\boldsymbol{R}_z(+\pi/2)\boldsymbol{\Sigma}\boldsymbol{U}^\mathsf{T} \qquad \boldsymbol{R}_1 = \boldsymbol{U}\boldsymbol{R}_z(+\pi/2)\boldsymbol{V}^\mathsf{T} \tag{13.19}$$

$$\boldsymbol{t}_2 = \boldsymbol{U}\boldsymbol{R}_z(-\pi/2)\boldsymbol{\Sigma}\boldsymbol{U}^\mathsf{T} \qquad \boldsymbol{R}_2 = \boldsymbol{U}\boldsymbol{R}_z(-\pi/2)\boldsymbol{V}^\mathsf{T} \tag{13.20}$$

where $\boldsymbol{E} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\mathsf{T}$ *is the singular value decomposition of the matrix* $\boldsymbol{E}$*, and* $\boldsymbol{R}_z(+\pi/2)$ *is an elementary rotation around the z-axis of an angle* $\pi/2$*.*



2-View Geometry limitation

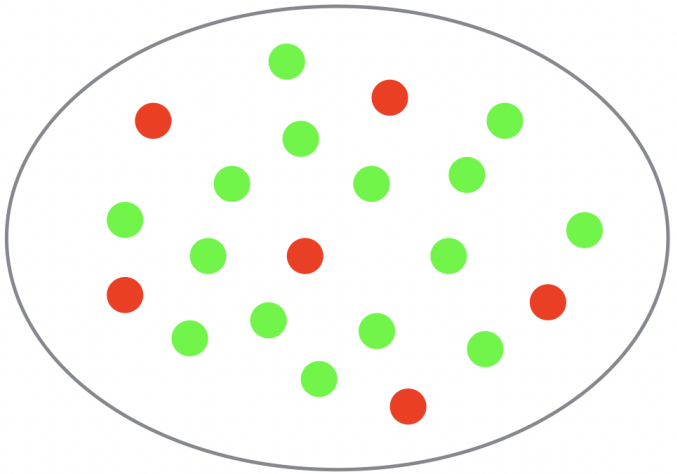


In practice:

- Many wrong correspondences (outliers)
- Some 3D points might be moving

Solution: RANSAC Algorithm

RANdom **SA**mple **C**onsensus



Problem: estimate model *P* from N data points, possibly corrupted with outliers.

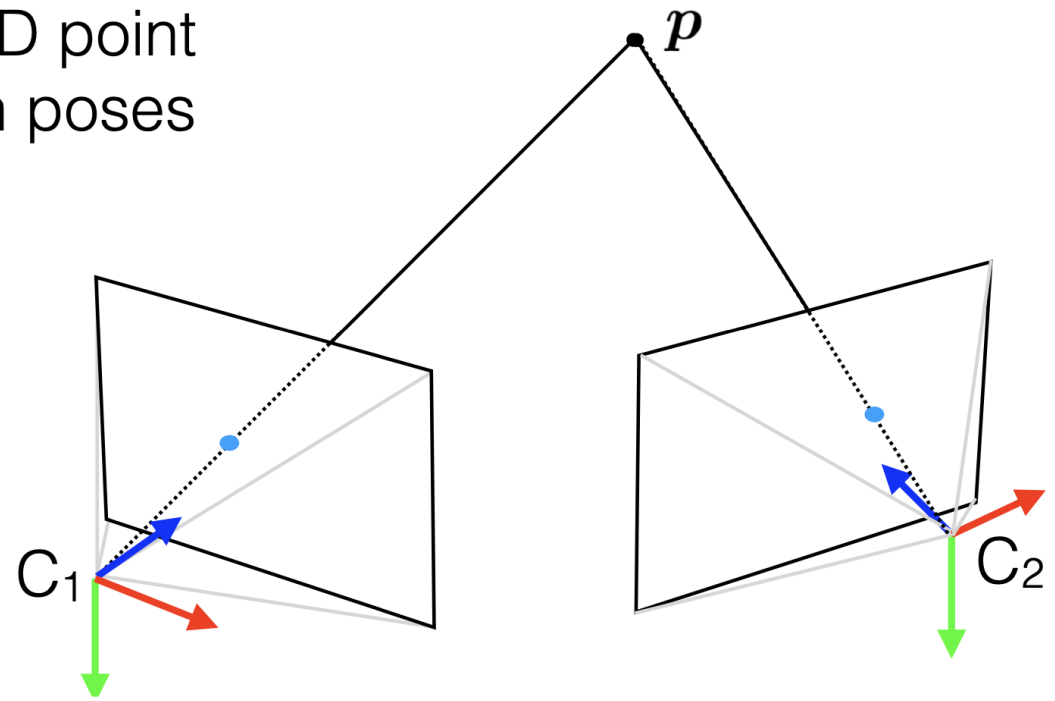
Assume: we have an algorithm to estimate *P* from *n* data points (n << N)

Basic idea:

1. sample *n* points
2. compute an estimate *P'* of *P*
3. count how many other points agree with *P'*
4. repeat until you get a *P'* that agrees with many points

Triangulation

Compute 3D point
from known poses

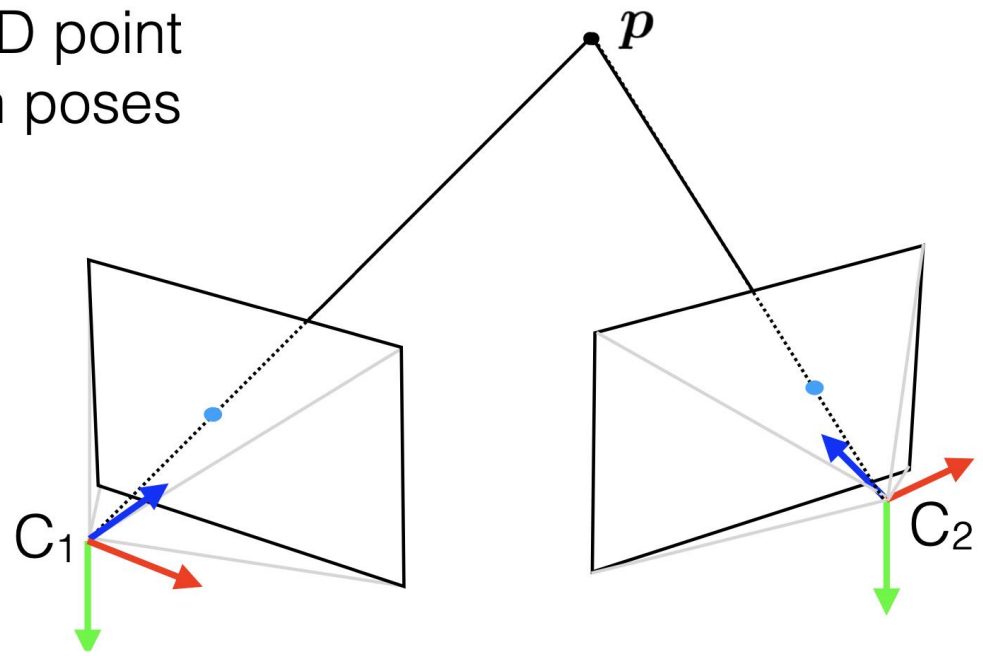


$$\lambda_1 \tilde{\boldsymbol{x}}_1 = \underbrace{\boldsymbol{K}_1[\boldsymbol{R}_{\mathrm{w}}^{\mathrm{c}1} \; \boldsymbol{t}_{\mathrm{w}}^{\mathrm{c}1}]}_{\boldsymbol{\Pi}_1} \tilde{\boldsymbol{p}}^{\mathrm{w}} \qquad \lambda_2 \tilde{\boldsymbol{x}}_2 = \underbrace{\boldsymbol{K}_2[\boldsymbol{R}_{\mathrm{w}}^{\mathrm{c}2} \; \boldsymbol{t}_{\mathrm{w}}^{\mathrm{c}2}]}_{\boldsymbol{\Pi}_2} \tilde{\boldsymbol{p}}^{\mathrm{w}}$$

Linear triangulation: $\min_{\|\tilde{\boldsymbol{p}}^{\mathrm{w}}\|=1} \|\boldsymbol{A}\tilde{\boldsymbol{p}}^{\mathrm{w}}\|^2$

Triangulation

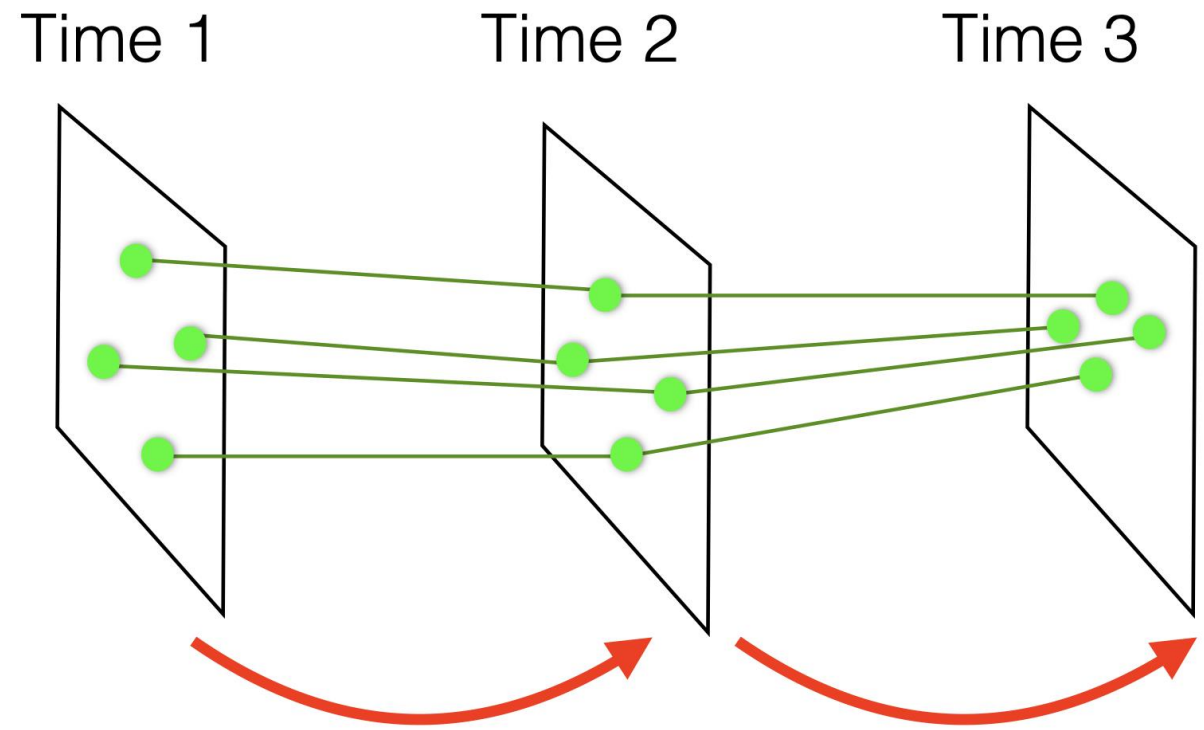
Compute 3D point
from known poses



$$\lambda_1 \tilde{\boldsymbol{x}}_1 = \underbrace{\boldsymbol{K}_1[\boldsymbol{R}_{\mathrm{w}}^{\mathrm{c}1} \ \boldsymbol{t}_{\mathrm{w}}^{\mathrm{c}1}]}_{\boldsymbol{\Pi}_1} \tilde{\boldsymbol{p}}^{\mathrm{w}} \qquad \lambda_2 \tilde{\boldsymbol{x}}_2 = \underbrace{\boldsymbol{K}_2[\boldsymbol{R}_{\mathrm{w}}^{\mathrm{c}2} \ \boldsymbol{t}_{\mathrm{w}}^{\mathrm{c}2}]}_{\boldsymbol{\Pi}_2} \tilde{\boldsymbol{p}}^{\mathrm{w}}$$

$$\min_{\boldsymbol{p}^{\mathrm{w}}} \|\boldsymbol{x}_1 - \pi(\boldsymbol{R}_{\mathrm{c}_1}^{\mathrm{w}}, \boldsymbol{t}_{\mathrm{c}_1}^{\mathrm{w}}, \boldsymbol{p}^{\mathrm{w}})\|^2 + \|\boldsymbol{x}_2 - \pi(\boldsymbol{R}_{\mathrm{c}_2}^{\mathrm{w}}, \boldsymbol{t}_{\mathrm{c}_2}^{\mathrm{w}}, \boldsymbol{p}^{\mathrm{w}})\|^2$$

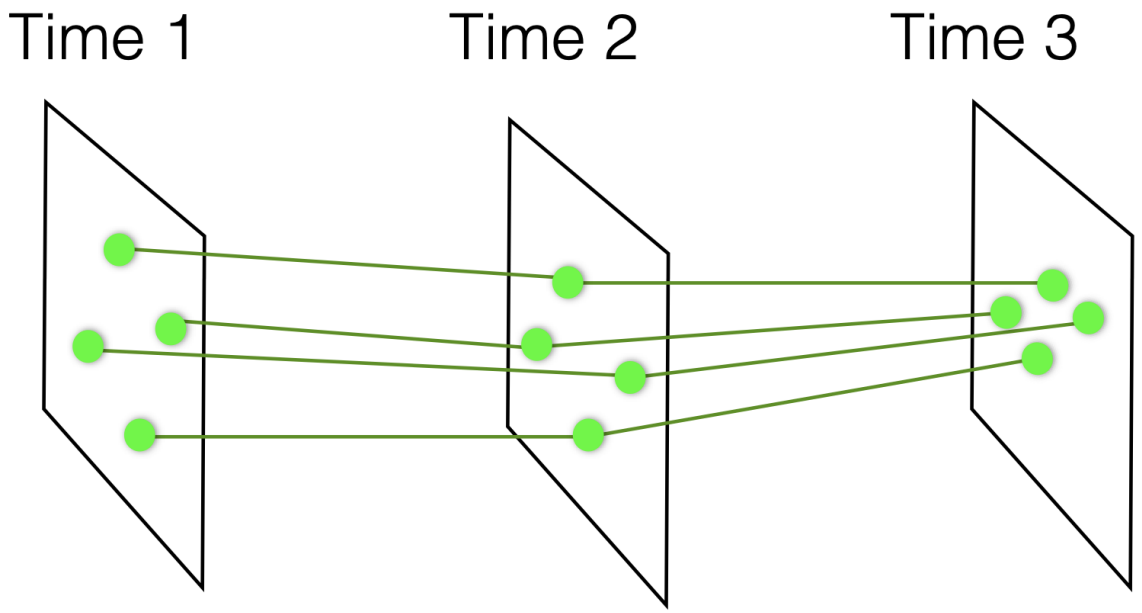
Motion Estimation



$$\boldsymbol{E}_{12} = \underset{\boldsymbol{E}_{12} \in \mathcal{S}_E}{\arg\min} \sum_{k=1}^{N} |\tilde{\boldsymbol{y}}_{k,2}^{\mathsf{T}} \boldsymbol{E}_{12} \tilde{\boldsymbol{y}}_{k,1}|^2$$

$$\boldsymbol{E}_{23} = \underset{\boldsymbol{E}_{23} \in \mathcal{S}_E}{\arg\min} \sum_{k=1}^{N} |\tilde{\boldsymbol{y}}_{k,3}^{\mathsf{T}} \boldsymbol{E}_{23} \tilde{\boldsymbol{y}}_{k,2}|^2$$

Motion And Structure Estimation



$$\min_{\substack{(\boldsymbol{R}_{\mathrm{c}_i}^{\mathrm{w}},\boldsymbol{t}_{\mathrm{c}_i}^{\mathrm{w}}),i=1,2,3\\ \boldsymbol{p}_k^{\mathrm{w}},k=1,\ldots,N}} \sum_{k=1}^{N}\sum_{i=1}^{3} \left\|\boldsymbol{x}_{k,i} - \pi(\boldsymbol{R}_{\mathrm{c}_i}^{\mathrm{w}}, \boldsymbol{t}_{\mathrm{c}_i}^{\mathrm{w}}, \boldsymbol{p}_k^{\mathrm{w}})\right\|^2$$

Generalizes to K cameras: **Bundle adjustment**

Place Recognition – Bag of Words

- Is pose estimation using visual (inertial) odometry enough?

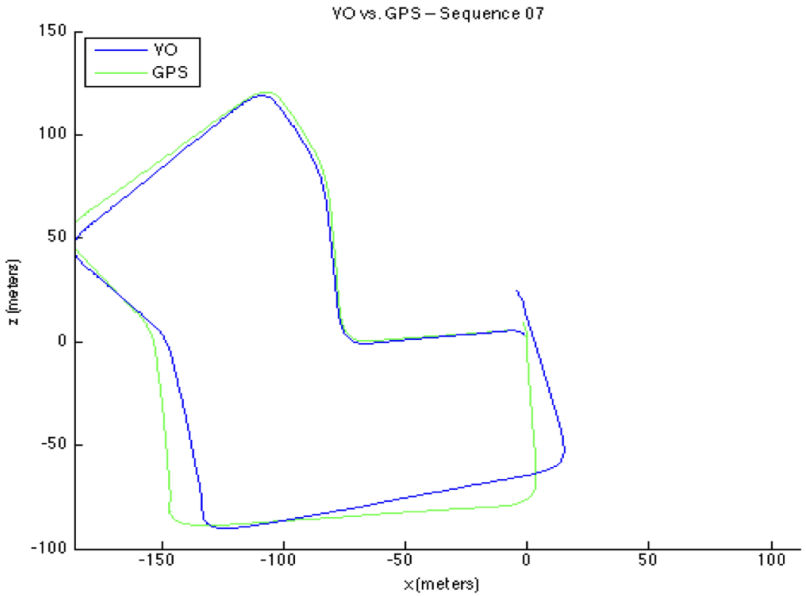
No! the error is accumulated over the time

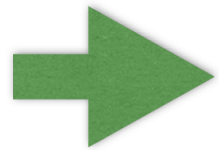
Solution:

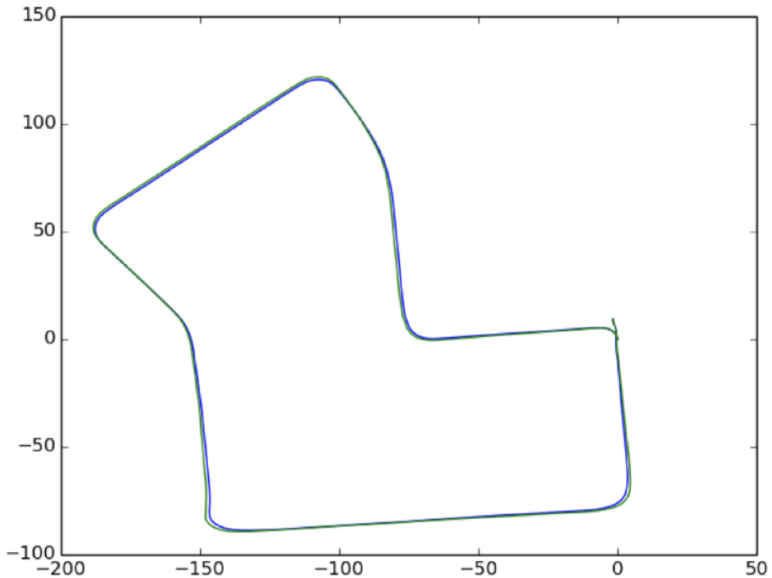
- Loop Closure
- Place recognition for lop closure!

Visual odometry

SLAM







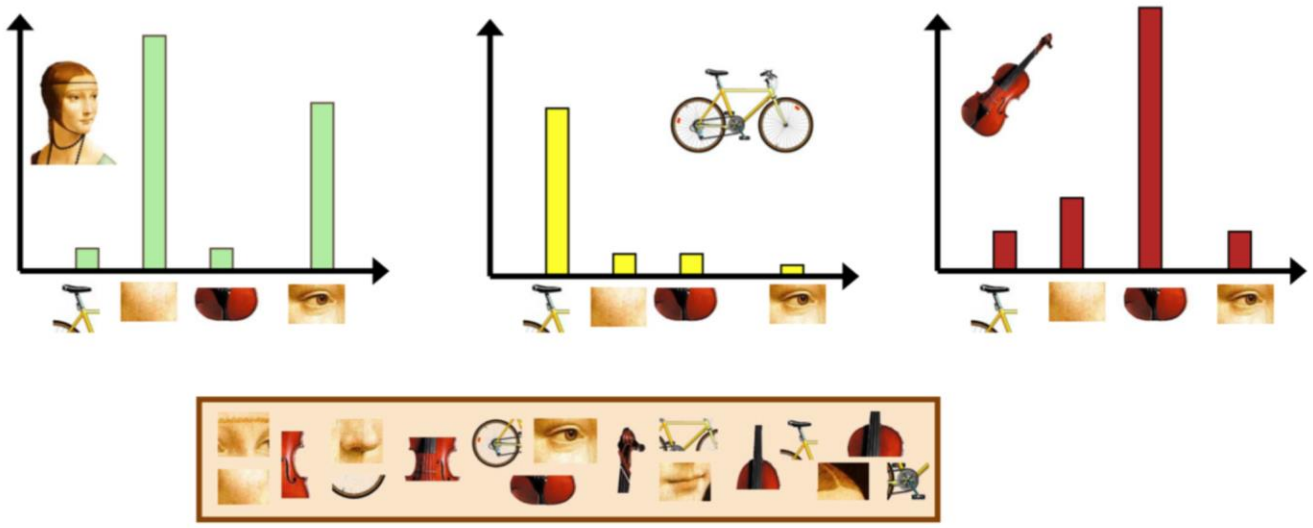
SLAM requires:

- place recognition => loop closure detection

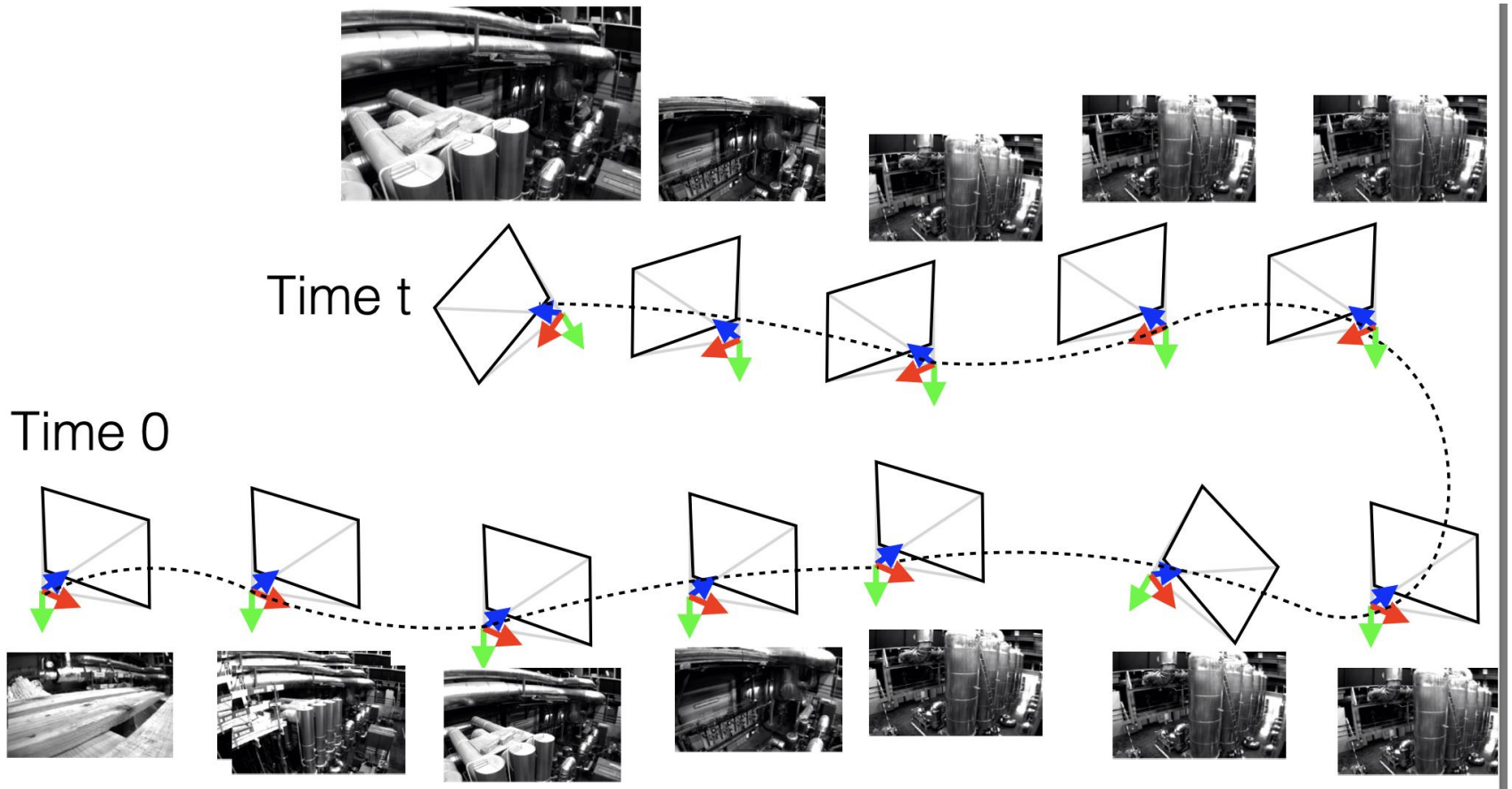
and / or

- Object detection => landmark detection

Bag of (visual) words



Place Recognition

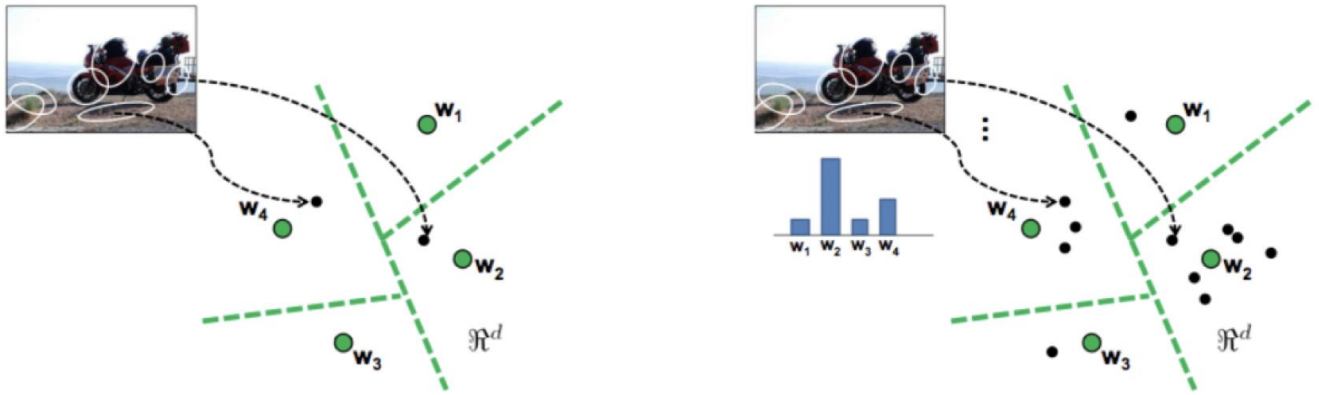


Does the image at time “t” picture a place seen in previous images?

Local Descriptors and Place Recognition

Based on text retrieval and summarization methods

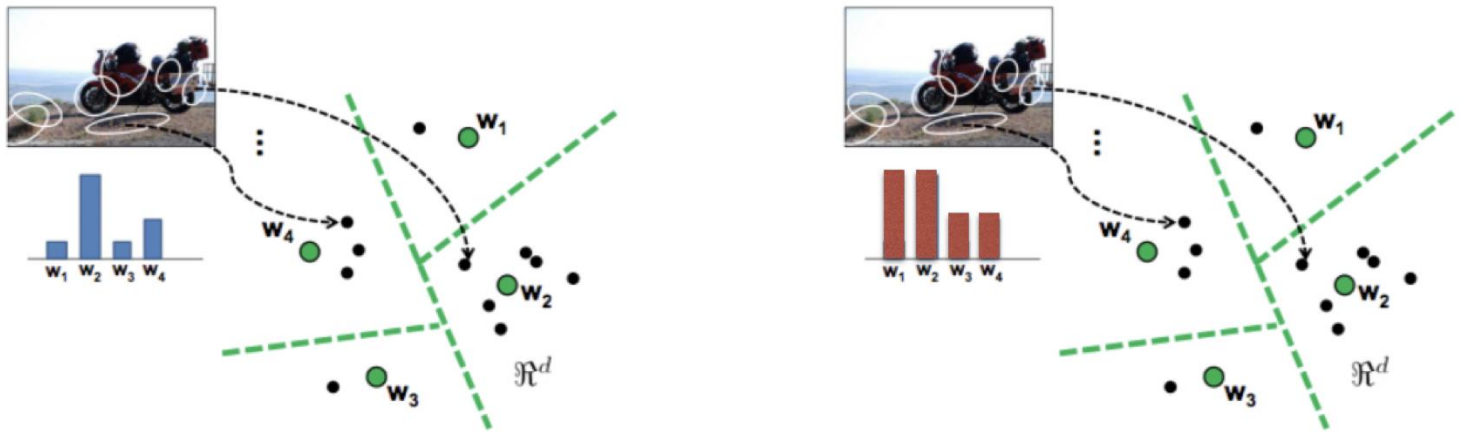
1) Extract features and descriptors in image
2) Discretize feature space (clustering)
3) Store the frequency of the features for each image



Each cluster is a “visual word”
Typically 5k-10k (up to 100k) visual words

J. Sivic and A. Zisserman. Video Google: A text retriev
approach to object matching in videos. In *ICCV*, 2003.

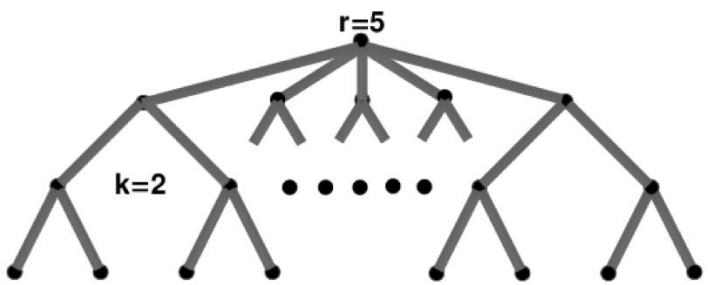
Local descriptors: Bag of Words



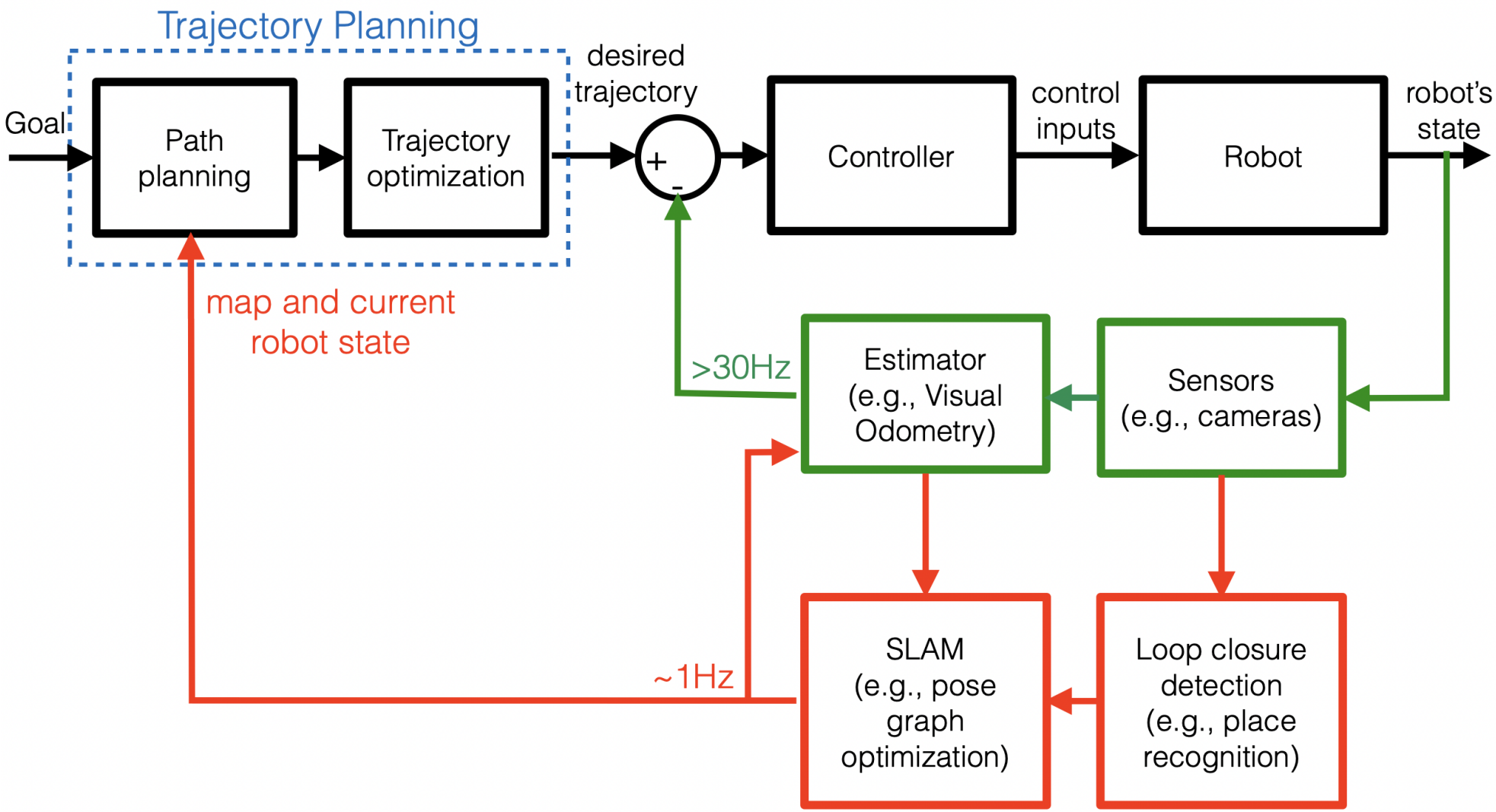
Two images are compared based on the corresponding histogram (Hamming distances, other metrics, …)

Faster version: **vocabulary tree**

Alternatives**: VLAD** (Vector of Locally Aggregated Descriptors), **Fisher vectors**

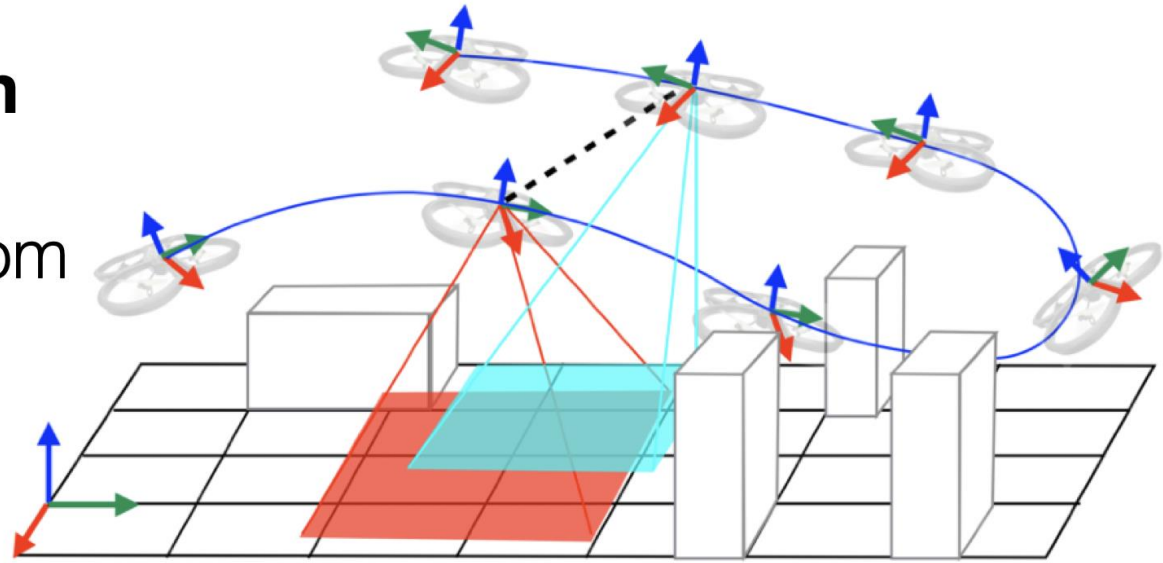


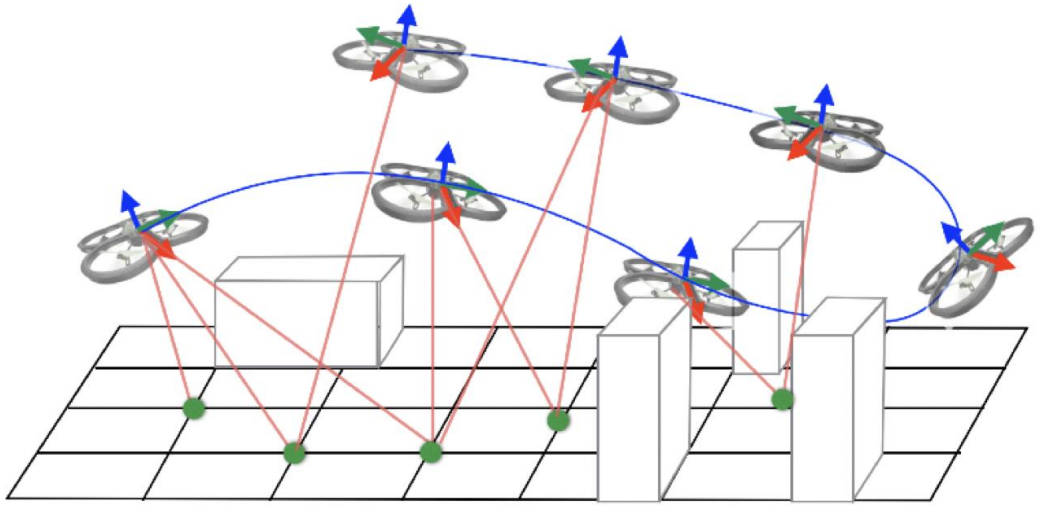
Big Picture


Trajectory Planning
Goal
Path planning
Trajectory optimization
desired trajectory
+
-
Controller
control inputs
Robot
robot's state
map and current robot state
>30Hz
Estimator (e.g., Visual Odometry)
Sensors (e.g., cameras)
~1Hz
SLAM (e.g., pose graph optimization)
Loop closure detection (e.g., place recognition)

Simultaneous Localization and Mapping

Pose Graph Optimization
(a.k.a. pose SLAM):
Estimate only trajectory from sensor data





Landmark-based SLAM:
Estimate trajectory of robot and position of external landmarks from sensor data